arXiv:1504.06952v21 [cs.LG] 15 Sep 2016

Random Forest for the Contextual Bandit Problem

Raphaël Féraud
raphael.feraud@orange.com
Orange Labs, France

Elys Allesiardo
elys.allesiardo@orange.com
Orange Labs, France

Tanguy Urvoy
tanguy.urvoy@orange.com
Orange Labs, France

Fabrice Clérot
fabrice.clerot@orange.com
Orange Labs, France

Abstract

To address the contextual bandit problem, we propose an online *random forest* algorithm. The analysis of the proposed algorithm is based on the sample complexity needed to find the optimal decision stump. Then, the decision stumps are recursively stacked in a random collection of decision trees, BANDIT FOREST. We show that the proposed algorithm is near optimal. The dependence of the sample complexity upon the number of contextual variables is logarithmic. The computational cost of the proposed algorithm with respect to the time horizon is linear. These analytical results allow the proposed algorithm to be efficient in real applications, where the number of events to process is huge, and where we expect that some contextual variables, chosen from a large set, have potentially non-linear dependencies with the rewards. In the experiments done to illustrate the theoretical analysis, BANDIT FOREST obtain promising results in comparison with state-of-the-art algorithms.

1 Introduction

By interacting with streams of events, machine learning algorithms are used for instance to optimize the choice of ads on a website, to choose the best human machine interface, to recommend products on a web shop, to insure self-care of set top boxes, to assign the best wireless network to mobile phones. With the now rising *internet of things*, the number of decisions (or actions) to be taken by more and more autonomous devices further increases. In order to control the cost and the potential risk to deploy a lot of machine learning algorithms in the long run, we need scalable algorithms which provide strong theoretical guarantees.

Appearing in Proceedings of the 19th International Conference on Artificial Intelligence and Statistics (AISTATS) 2016, Cadiz, Spain. JMLR: W&CP volume 41. Copyright 2016 by the authors.

Most of these applications necessitate to take and optimize decisions with a partial feedback. Only the reward of the chosen decision is known. Does the user click on the proposed ad ? The most relevant ad is never revealed. Only the click on the proposed ad is known. This well known problem is called *multi-armed bandits* (MAB). In its most basic formulation, it can be stated as follows: there are K decisions, each having an unknown distribution of bounded rewards. At each step, one has to choose a decision and receives a reward. The performance of a MAB algorithm is assessed in terms of *regret* (or opportunity loss) with regards to the unknown optimal decision. Optimal solutions have been proposed to solve this problem using a stochastic formulation in Auer et al (2002), using a Bayesian formulation in Kaufman et al (2012), or using an adversarial formulation in Auer et al (2002). While these approaches focus on the minimization of the expected regret, the PAC setting (see Vailant (1984)) or the (ϵ, δ)-best-arm identification, focuses on the sample complexity (i.e. the number of time steps) needed to find an ϵ-approximation of the best arm with a failure probability of δ. This formulation has been studied for MAB problem in Even-Dar et al (2002); Bubeck et al (2009), for dueling bandit problem in Urvoy et al (2013), and for linear bandit problem in Soare et al (2014).

Several variations of the MAB problem have been introduced in order to fit practical constraints coming from ad serving or marketing optimization. These variations include for instance the death and birth of arms in Chakrabarti et al (2008), the availability of actions in Kleinberg et al (2008) or a drawing without replacement in Féraud and Urvoy (2012, 2013). But more importantly, in most of these applications, a rich contextual information is also available. For instance, in ad-serving optimization we know the web page, the position in the web page, or the profile of the user. This contextual information must be exploited in order to decide which ad is the most relevant to display. Following Langford and Zhang (2007); Dudik et al (2011), in order to analyze the proposed algorithms, we formalize below the *contextual bandit problem* (see Algorithm 1).

Let $\mathbf{x_t} \in \{0,1\}^M$ be a vector of binary values describing the environment at time t. Let $\mathbf{y_t} \in [0,1]^K$ be a vec-

tor of bounded rewards at time t, and $y_{k_t}(t)$ be the reward of the action (decision) k_t at time t. Let $D_{x,y}$ be a joint distribution on $(\mathbf{x}, \mathbf{y})$. Let $\mathcal{A}$ be a set of K actions. Let $\pi : \{0,1\}^M \to \mathcal{A}$ be a policy, and Π the set of policies.

Algorithm 1 The contextual bandit problem

repeat
  $(\mathbf{x_t}, \mathbf{y_t})$ is drawn according to $D_{x,y}$
  $\mathbf{x_t}$ is revealed to the player
  The player chooses an action $k_t = \pi_t(\mathbf{x_t})$
  The reward $y_{k_t}(t)$ is revealed
  The player updates its policy π_t
  $t = t + 1$
until $t = T$

Notice that this setting can easily be extended to categorical variables through a binary encoding. The optimal policy π^* maximizes the expected gain:

$$\pi^* = \arg\max_{\pi \in \Pi} \mathbb{E}_{D_{x,y}} \left[y_{\pi(\mathbf{x_t})} \right]$$

Let $k_t^* = \pi^*(\mathbf{x_t})$ be the action chosen by the optimal policy at time t. The performance of the player policy is assessed in terms of expectation of the accumulated regret against the optimal policy with respect to $D_{x,y}$:

$$\mathbb{E}_{D_{x,y}} \left[R(T) \right] = \sum_{t=1}^{T} \mathbb{E}_{D_{x,y}} \left[y_{k_t^*}(t) - y_{k_t}(t) \right] ,$$

where $R(T)$ is the accumulated regret at time horizon T.

2 Our contribution

Decision trees work by partitioning the input space in hyper-boxes. They can be seen as a combination of rules, where only one rule is selected for a given input vector. Finding the optimal tree structure (i.e. the optimal combination of rules) is NP-hard. For this reason, a greedy approach is used to build the decision trees offline (see Breiman et al (1984)) or online (see Domingos and Hulten (2000)). The key concept behind the greedy decision trees is the decision stump. While Monte-Carlo Tree Search approaches (see Kocsis and Szépesvari (2006)) focus on the regret minimization (i.e. maximization of gains), the analysis of proposed algorithms is based on the sample complexity needed to find the optimal decision stump with high probability. This formalization facilitates the analysis of decision tree algorithms. Indeed, to build a decision tree under limited resources, one needs to eliminate most of possible branches. The sample complexity is the stopping criterion we need to stop exploration of unpromising branches. In BANDIT FOREST, the decision stumps are recursively stacked in a random collection of L decision trees of maximum depth D. We show that BANDIT FOREST algorithm is near optimal with respect to a strong reference: a *random forest* built knowing the joint distribution of the contexts and rewards.

In comparison to algorithms based on search of the best policy from a finite set of policies (see Auer et al (2002); Dudik et al (2011); Agrawal et al (2014)) our approach has several advantages. First, we take advantage of the fact that we know the structure of the set of policies to obtain a linear computational cost with respect to the time horizon T. Second, as our approach does not need to store a weight for each possible tree, we can use deeper rules without exceeding the memory resources. In comparison to the approaches based on a linear model (see LINUCB in Li et al (2010)), our approach also has several advantages. First, it is better suited for the case where the dependence between the rewards and the contexts is not linear. Second, the dependence of regret bounds of the proposed algorithms on the number of contextual variables is in the order of $O(\log M)$ while the one of linear bandits is in $O(\sqrt{M})$ [1]. Third, its computational cost with respect to time horizon in $O(LMDT)$ allows to process large set of variables, while linear bandits are penalized by the update of a $M \times M$ matrix at each update, which leads to a computational cost in $O(KM^2T)$.

3 The decision stump

In this section, we consider a model which consists of a decision stump based on the values of a single contextual variable, chosen from at set of M binary variables.

3.1 A gentle start

In order to explain the principle and to introduce the notations, before describing the decision stump used to build BANDIT FOREST, we illustrate our approach on a toy problem. Let k_1 and k_2 be two actions. Let x_{i_1} and x_{i_2} be two binary random variables, describing the context. In this illustrative example, the contextual variables are assumed to be independent. Relevant probabilities and rewards are summarized in Table 1. $\mu_{k_2}^{i_1}|v$ denotes the conditional expected reward of the action k_2 given $x_{i_1} = v$, and $P(x_{i_1} = v)$ denotes the probability to observe $x_{i_1} = v$.

We compare the strategies of different players. Player 1 only uses uncontextual expected rewards, while Player 2 uses the knowledge of x_{i_1} to decide. According to Table 1, the best strategy for Player 1 is to always choose action k_2. His expected reward will be $\mu_{k_2} = 7/16$. Player 2 is able to adapt his strategy to the context: his best strategy is to choose k_2 when $x_{i_1} = v_0$ and k_1 when $x_{i_1} = v_1$.

[1] In fact the dependence on the number of contextual variables of the gap dependent regret bound is in $O(M^2)$ (see Theorem 5 in Abbasi-Yadkori and Szepesvári (2011)).

Table 1: The mean reward of actions k_1 and k_2 knowing each context value, and the probability to observe this context.

	v_0	v_1
$\mu^{i_1}_{k_1}\|v$	0	1
$\mu^{i_1}_{k_2}\|v$	3/5	1/6
$P(x_{i_1}=v)$	5/8	3/8
$\mu^{i_2}_{k_1}\|v$	1/4	3/4
$\mu^{i_2}_{k_2}\|v$	9/24	5/8
$P(x_{i_2}=v)$	3/4	1/4

According to Table 1, his expected reward will be:

$$\begin{aligned}\mu^{i_1} &= P(x_{i_1}=v_0)\cdot\mu^{i_1}_{k_2}|v_0 + P(x_{i_1}=v_1)\cdot\mu^{i_1}_{k_1}|v_1\\ &= \mu^{i_1}_{k_2,v_0} + \mu^{i_1}_{k_1,v_1} = 3/4\,,\end{aligned}$$

where $\mu^{i_1}_{k_2,v_0}$ and $\mu^{i_1}_{k_1,v_1}$ denote respectively the expected reward of the action k_2 and $x_{i_1}=v_0$, and the expected reward of the action k_1 and $x_{i_1}=v_1$. Whatever the expected rewards of each value, the player, who uses this knowledge, is the expected winner. Indeed, we have:

$$\mu^i = \max_k \mu^i_{k,v_0} + \max_k \mu^i_{k,v_1} \geq \max_k \mu_k$$

Now, if a third player uses the knowledge of the contextual variable x_{i_2}, his expected reward will be:

$$\mu^{i_2} = \mu^{i_2}_{k_2,v_0} + \mu^{i_2}_{k_1,v_1} = 15/32$$

Player 2 remains the expected winner, and therefore x_{i_1} is the best contextual variable to decide between k_1 and k_2.

The best contextual variable is the one which maximizes the expected reward of best actions for each of its values. We use this principle to build a reward-maximizing decision stump.

3.2 Variable selection

Let $\mathcal{V}$ be the set of variables, and $\mathcal{A}$ be the set of actions. Let $\mu^i_k|v = \mathbb{E}_{D_y}[y_k \cdot \mathbb{1}_{x_i=v}]$ be the expected reward of the action k conditioned to the observation of the value v of the variable x_i. Let $\mu^i_{k,v} = P(v).\mu^i_k|v = \mathbb{E}_{D_{x,y}}[y_k \cdot \mathbb{1}_{x_i=v}]$ be the expected reward of the action k and the value v of the binary variable x_i. The expected reward when using variable x_i to select the best action is the sum of expected rewards of the best actions for each of its possible values:

$$\mu^i = \sum_{v\in\{0,1\}} P(v).\max_k \mu^i_k|v = \sum_{v\in\{0,1\}} \max_k \mu^i_{k,v}$$

The optimal variable to be used for selecting the best action is: $i^* = \arg\max_{i\in\mathcal{V}} \mu^i$.

The algorithm VARIABLE SELECTION chooses the best variable. The Round-robin function sequentially explores the actions in $\mathcal{A}$ (see Algorithm 2 line 4). Each time t_k the reward of the selected action k is unveiled, the estimated expected rewards of the played action k and observed values $\hat{\mu}^i_{k,v}$ and all the estimated rewards of variables $\hat{\mu}^i$ are updated (see VE function lines 2-7). This *parallel exploration strategy* allows the algorithm to explore efficiently the variable set. When all the actions have been played once (see VE function line 8), irrelevant variables are eliminated if:

$$\hat{\mu}^{i'} - \hat{\mu}^i + \epsilon \geq 4\sqrt{\frac{1}{2t_k}\log\frac{4KMt_k^2}{\delta}}\,, \qquad (1)$$

where $i' = \arg\max_i \hat{\mu}^i$, and t_k is the number of times the action k has been played.

Algorithm 2 VARIABLE SELECTION

Inputs: $\epsilon \in [0,1)$, $\delta \in (0,1]$
Output: an ϵ-approximation of the best variable

1: $t=0, \forall k\ t_k=0, \forall(i,k,v)\ \hat{\mu}^i_{k,v}=0, \forall i\ \hat{\mu}^i=0$
2: **repeat**
3: Receive the context vector $\mathbf{x_t}$
4: Play $k = \text{Round-robin}\,(\mathcal{A})$
5: Receive the reward $y_k(t)$
6: $t_k = t_k + 1$
7: $\mathcal{V}$=VE$(t_k, k, \mathbf{x_t}, y_k, \mathcal{V}, \mathcal{A})$
8: $t = t+1$
9: **until** $|\mathcal{V}| = 1$

1: **Function** VE$(t, k, \mathbf{x_t}, y_k, \mathcal{V}, \mathcal{A})$
2: **for** each remaining variable $i \in \mathcal{V}$ **do**
3: **for** each value v **do**
4: $\hat{\mu}^i_{k,v} = \frac{y_k}{t}\mathbb{1}_{x_i=v} + \frac{t-1}{t}\hat{\mu}^i_{k,v}$
5: **end for**
6: $\hat{\mu}^i = \sum_{v\in\{0,1\}} \max_k \hat{\mu}^i_{k,v}$
7: **end for**
8: **if** $k = \text{LastAction}(\mathcal{A})$ **then**
9: Remove irrelevant variables from $\mathcal{V}$ according to equation 1, or 3
10: **end if**
11: **return** $\mathcal{V}$

The parameter $\delta \in (0,1]$ corresponds to the probability of failure. The use of the parameter ϵ comes from practical reasons. The parameter ϵ is used in order to tune the convergence speed of the algorithm. In particular, when two variables provide the highest expected reward, the use of $\epsilon > 0$ ensures that the algorithm stops. The value of ϵ has to be in the same order of magnitude as the best mean reward we want to select. In the analysis of algorithms, we will consider the case where $\epsilon = 0$. Lemma 1 analyzes the sample complexity (the number of iterations before stopping) of VARIABLE SELECTION.

Lemma 1: *when $K \geq 2$, $M \geq 2$, and $\epsilon = 0$, the sample complexity of* VARIABLE SELECTION *needed to obtain* $\mathbb{P}(i' \neq i^*) \leq \delta$ *is:*

$$t^* = \frac{64K}{\Delta_1^2} \log \frac{8KM}{\delta \Delta_1}, \text{ where } \quad \Delta_1 = \min_{i \neq i^*} (\mu^{i^*} - \mu^i)$$

Lemma 1 shows that the dependence of the sample complexity needed to select the optimal variable is in $O(\log M)$. This means that VARIABLE SELECTION can be used to process large set of contextual variables, and hence can be easily extended to categorical variables, through a binary encoding with only a logarithmic impact on the sample complexity. Finally, Lemma 2 shows that the variable selection algorithm is optimal up to logarithmic factors.

Lemma 2: *There exists a distribution $D_{x,y}$ such that any algorithm finding the optimal variable i^* has a sample complexity of at least:*

$$\Omega\left(\frac{K}{\Delta_1^2} \log \frac{1}{\delta}\right)$$

3.3 Action selection

To complete a decision stump, one needs to provide an algorithm which optimizes the choice of the best action knowing the selected variable. Any stochastic bandit algorithm such as UCB (see Auer et al (2002)), TS (see Thompson (1933); Kaufman et al (2012)) or BESA (see Baransi et al (2014)) can be used. For the consistency of the analysis, we choose SUCCESSIVE ELIMINATION in Even-Dar et al (2002, 2006) (see Algorithm 3), that we have renamed ACTION SELECTION. Let $\mu_k = \mathbb{E}_{D_y}[y_k]$ be the expected reward of the action k taken with respect to D_y. The estimated expected reward of the action is denoted by $\hat{\mu}_k$.

Algorithm 3 ACTION SELECTION

Inputs: $\epsilon \in [0, 1)$, $\delta \in (0, 1]$
Output: an ϵ-approximation of the best arm

$t = 0$, $\forall k\ \hat{\mu}_k = 0$ and $t_k = 0$
repeat
  Play $k = \text{Round-robin}(A)$
  Receive the reward $y_k(t)$
  $t_k = t_k + 1$
  $\mathcal{A}$=AE$(t_k, k, y_k(t), \mathcal{A})$
  $t = t + 1$
until $|\mathcal{A}| = 1$

The irrelevant actions in the set $\mathcal{A}$ are successively eliminated when:

$$\hat{\mu}_{k'} - \hat{\mu}_k + \epsilon \geq 2\sqrt{\frac{1}{2t_k} \log \frac{4Kt_k^2}{\delta}}\,, \tag{2}$$

Function AE$(t, k, \mathbf{x_t}, y_k, \mathcal{A})$
$\hat{\mu}_k = \frac{y_k}{t} + \frac{t-1}{t}\hat{\mu}_k$
if k=LastAction($\mathcal{A}$) **then**
  Remove irrelevant actions from $\mathcal{A}$ according to equation 2, or 4
end if
return $\mathcal{A}$

where $k' = \arg\max_k \hat{\mu}_k$, and t_k is the number of times the action k has been played.

Lemma 3: *when $K \geq 2$, and $\epsilon = 0$, the sample complexity of* ACTION SELECTION *needed to obtain* $\mathbb{P}(k' \neq k^*) \leq \delta$ *is:*

$$t^* = \frac{64K}{\Delta_2^2} \log \frac{4K}{\delta \Delta_2}, \text{ where } \quad \Delta_2 = \min_k (\mu_{k^*} - \mu_k).$$

The proof of Lemma 3 is the same than the one provided for SUCCESSIVE ELIMINATION in Even-Dar et al (2006). Finally, Lemma 4 states that the action selection algorithm is optimal up to logarithmic factors (see Mannor and Tsitsiklis (2004) Theorem 1 for the proof).

Lemma 4: *There exists a distribution $D_{x,y}$ such that any algorithm finding the optimal action k^* has a sample complexity of at least:*

$$\Omega\left(\frac{K}{\Delta_2^2} \log \frac{1}{\delta}\right)$$

3.4 Analysis of a decision stump

The decision stump uses the values of a contextual variable to choose the actions. The optimal decision stump uses the best variable to choose the best actions. It plays at time t: $k_t^* = \arg\max_k \mu_k^{i^*}|v$, where $i^* = \arg\max_i \mu^i$, and $v = x_{i^*}(t)$. The expected gain of the optimal policy is:

$$\mathbb{E}_{D_{x,y}}\left[y_{k_t^*}(t)\right] = \sum_v P(v)\mu_{k^*}^{i^*}|v = \sum_v \mu_{k^*,v}^{i^*} = \mu^{i^*}$$

To select the best variable, one needs to find the best action of each value of each variable. In DECISION STUMP algorithm (see Algorithm 4), an action selection task is allocated for each value of each contextual variable. When the reward is revealed, all the estimated rewards of variables $\hat{\mu}^i$ and the estimated rewards of the played action knowing the observed values of variables $\hat{\mu}_k^i|v$ are updated (respectively in VE and AE functions): the variables and the actions are simultaneously explored. However, the elimination of actions becomes effective only when the best variable is selected. Indeed, if an action k is eliminated for a value v_0 of a variable i, the estimation of $\hat{\mu}_{k,v_1}^i$, the mean expected reward of the action k for the value v_1, is biased. As a consequence, if an action is eliminated during the exploration

of variables, the estimation of the mean reward μ^i can be biased. That is why, the lower bound of decision stump problem is the sum of lower bound of variable and action selection problems (see Theorem 2). The only case where an action can be eliminated before the best variable be selected, is when this action is eliminated for all values of all variables. For simplicity of the exposition of the DECISION STUMP algorithm we did not handle this case here.

Algorithm 4 DECISION STUMP

$t = 0$, $\forall k\ t_k = 0$, $\forall i\ \hat{\mu}^i = 0$, $\forall (i,v,k)\ t_{i,v,k} = 0$, $\mathcal{A}_{i,v} = \mathcal{A}$, $\hat{\mu}^i_{k,v} = 0$ and $\hat{\mu}^i_k|v = 0$
repeat
  Receive the context vector $\mathbf{x_t}$
  if $|\mathcal{V}| > 1$ **then** Play $k = \text{Round-robin}\,(\mathcal{A})$
  else Play $k = \text{Round-Robin}\,(\mathcal{A}_{i,v})$
  Receive the reward $y_k(t)$
  $t_k = t_k + 1$
  $\mathcal{V}$=VE$(t_k, k, \mathbf{x_t}, y_k, \mathcal{V}, \mathcal{A})$
  for each variable $i \in \mathcal{V}$ **do**
    $v = x_i(t)$, $t_{i,v,k} = t_{i,v,k} + 1$
    $\mathcal{A}_{i,v} = \text{AE}(t_{i,v,k}, k, \mathbf{x_t}, y_k, \mathcal{A}_{i,v})$
  end for
  $t = t + 1$
until $t = T$

Theorem 1: *when $K \geq 2$, $M \geq 2$, and $\epsilon = 0$, the sample complexity needed by* DECISION STUMP *to obtain $\mathbb{P}\left(i' \neq i^*\right) \leq \delta$ and $\mathbb{P}\left(k'_t \neq k^*_t\right) \leq \delta$ is:*

$$t^* = \frac{64K}{\Delta_1^2} \log \frac{4KM}{\delta \Delta_1} + \frac{64K}{\Delta_2^2} \log \frac{4K}{\delta \Delta_2},$$

$$\textit{where } \Delta_1 = \min_{i \neq i^*} \left(\mu^{i^*} - \mu^i\right),$$
$$\textit{and } \Delta_2 = \min_{k \neq k^*, v \in \{0,1\}} \left(\mu^{i^*}_{k^*,v} - \mu^{i^*}_{k,v}\right).$$

Theorem 2 provides a lower bound for the sample complexity, showing that the factors $1/\Delta_1^2$ and $1/\Delta_2^2$ are inherent of the decision stump problem. Notice that for the linear bandit problem, the same factor $1/\Delta^2$ was obtained in the lower bound (see Lemma 2 in Soare et al (2014)).

Theorem 2: *It exists a distribution $D_{x,y}$ such that any algorithm finding the optimal decision stump has a sample complexity of at least:*

$$\Omega\left(\left(\frac{1}{\Delta_1^2} + \frac{1}{\Delta_2^2}\right) K \log \frac{1}{\delta}\right)$$

Remark 1: The factors in $\log 1/\Delta_1$, $\log 1/\Delta_2$ and $\log 1/\Delta$ could vanish in Theorem 1 following the same approach as that *Median Elimination* in Even-Dar et al (2006). Despite the optimality of this algorithm, we did not choose it for the consistency of the analysis. Indeed, as it suppresses $1/4$ of variables (or actions) at the end of each elimination phase, *Median Elimination* is not well suited when a few number of variables (or actions) provide lot of rewards and the others not. In this case this algorithm spends a lot of times to eliminate non-relevant variables. This case is precisely the one, where we would like to use a local model such as a decision tree.

Remark 2: The extension of the analytical results to an ϵ-approximation of the best decision stump is straightforward using $\Delta_\epsilon = \max(\epsilon, \Delta)$.

4 BANDIT FOREST

A decision stump is a weak learner which uses only one variable to decide the best action. When one would like to combine D variables to choose the best action, a tree structure of $\binom{M}{D}.2^D$ multi-armed bandit problems has to be allocated. To explore and exploit this tree structure with limited memory resources, our approach consists in combining greedy decision trees. When decision stumps are stacked in a greedy tree, they combine variables in a greedy way to choose the action. When a set of randomized trees vote, they combine variables in a non-greedy way to choose the action.

As highlighted by empirical studies (see for instance Fernàndez-Delgado et al (2014)), *random forests* of Breiman (2001) have emerged as a serious competitors to state-of-the-art methods for classification tasks. In Biau (2012), the analysis of *random forests* shows that the reason of these good performances comes from the fact that the convergence of its learning procedure is consistent, and its rate of convergence depends only on the number of strong features, which is assumed to be low in comparison to the number of features. In this section, we propose to use a *random forest* built with the knowledge of $D_{x,y}$ as a reference for the proposed algorithm BANDIT FOREST.

Algorithm 5 θ-OGT $\left(c^*_\theta, d_\theta, \mathcal{V}_{c^*_\theta}\right)$

Inputs: $\theta \in [0,1]$
Output: the θ-optimal greedy tree when θ-OGT $((), 0, \mathcal{V})$ is called

if $d_\theta < D_\theta$ **then**
  $\mathcal{S}_{c^*_\theta} = \left\{i \in \mathcal{V}_{c^*_\theta} : f(\theta, c^*_\theta, i) = 1\right\}$
  $i^*_{d_\theta+1} = \arg\max_{i \in \mathcal{S}_{c^*_\theta}} \mu^i | c^*_\theta$
  θ-OGT $\left(c^*_\theta + (x_{i^*_{d_\theta+1}} = 0), d_\theta + 1, \mathcal{V}_{c^*_\theta} \setminus \{i^*_{d_\theta+1}\}\right)$
  θ-OGT $\left(c^*_\theta + (x_{i^*_{d_\theta+1}} = 1), d_\theta + 1, \mathcal{V}_{c^*_\theta} \setminus \{i^*_{d_\theta+1}\}\right)$
  else $k^*|c^*_\theta = \arg\max_k \mu_k | c^*_\theta$
end if

Let Θ be a random variable, which is independent of $\mathbf{x}$ and $\mathbf{y}$. Let $\theta \in [0,1]$ the value of Θ. Let D_θ be the maximal

depth of the tree θ. Let $c_\theta = \{(x_{i_1} = v), ..., (x_{i_{d_\theta}} = v)\}$ be the index of a path of the tree θ. We use $c_\theta(\mathbf{x_t})$ to denote the path c_θ selected at time t by the tree θ. Let $f(\theta, i, j) : [0,1] \times \{0,1\}^{2^D} \times M \rightarrow \{0,1\}$ be a function which parametrizes $\mathcal{V}_{c_\theta}$ the sets of available variables for each of 2^{D_θ} splits. We call θ-optimal greedy tree, the greedy tree built with the knowledge of $D_{x,y}$ and conditioned to the value θ of the random variable Θ (see Algorithm 5). We call optimal *random forest* of size L, a *random forest*, which consists of a collection of L θ-optimal greedy trees. At each time step, the optimal *random forest* chooses the action k_t^*, which obtains the higher number of votes:

$$k_t^* = \arg\max_k \sum_{i=1}^{L} \mathbb{1}_{k^*_{\theta_i,t}=k} \, ,$$

where $k^*_{\theta_i,t}$ is the action chosen by the optimal greedy tree θ_i at time t.

Algorithm 6 BANDIT FOREST

1: $t = 0, \forall \theta \; c_\theta = ()$ and $d_\theta = 0, \forall \theta$ NewPath$(c_\theta, \mathcal{V})$
2: **repeat**
3: Receive the context vector $\mathbf{x_t}$
4: **for** each θ **do** select the context path $c_\theta(\mathbf{x_t})$
5: **if** $\forall \theta \; d_\theta = D_\theta$ and $|\mathcal{S}_{c_\theta}| = 1$, and $\forall (\theta, i, v)$ $|\mathcal{A}_{c_\theta,i,v}| = 1$ **then** $k = \arg\max_k \sum_{i=1}^{L} \mathbb{1}_{k_{\theta_i,t}=k}$
6: **else** $k =$ Round-robin$(\mathcal{A})$
7: **endif**
8: Receive the reward $y_k(t)$
9: **for** each θ **do**
10: $t_{c_\theta,k} = t_{c_\theta,k} + 1$
11: $\mathcal{S}_{c_\theta}$=VE$(t_{c_\theta,k}, k, \mathbf{x_t}, y_k, \mathcal{S}_{c_\theta}, \mathcal{A})$
12: **for** each remaining variable i **do**
13: $v = x_i(t), t_{c_\theta,i,v,k} = t_{c_\theta,i,v,k} + 1$
14: $\mathcal{A}_{c_\theta,i,v} =$ AE$(t_{c_\theta,i,v,k}, k, \mathbf{x_t}, y_k, \mathcal{A}_{c_\theta,i,v})$
15: **end for**
16: **if** $|\mathcal{S}_{c_\theta}| = 1$ **and** $d_\theta < D_\theta$ **then**
17: $\mathcal{V}_{c_\theta} = \mathcal{V}_{c_\theta} \setminus \{i\}$
18: NewPath$(c_\theta + (x_i = 0), \mathcal{V}_{c_\theta})$
19: NewPath$(c_\theta + (x_i = 1), \mathcal{V}_{c_\theta})$
20: **end if**
21: **end for**
22: $t = t + 1$
23: **until** $t = T$

1: **Function** NewPath$(c_\theta, \mathcal{V}_{c_\theta})$
2: $\mathcal{S}_{c_\theta} = \{i \in \mathcal{V}_{c_\theta} : f(\theta, c_\theta, i) = 1\}$
3: $d_\theta = d_\theta + 1, \forall (i, v) \; \mathcal{A}_{c_\theta,i,v} = \mathcal{A}$
4: $\forall k \; t_{c_\theta,k} = 0, \forall (i, v, k) \; t_{c_\theta,i,v,k} = 0, \forall i \; \hat{\mu}^i | c_\theta = 0$
5: $\forall (i, k, v) \; \hat{\mu}^i_{k,v} | c_\theta = 0$ and $\hat{\mu}^i_k | (v, c_\theta) = 0$

BANDIT FOREST algorithm explores and exploits a set of L decision trees knowing $\theta_1, ..., \theta_L$ (see Algorithm 6). When a context $\mathbf{x_t}$ is received (line 3):

- For each tree θ, the path c_θ is selected (line 4).
- An action k is selected:
  - If the learning of all paths c_θ is finished, then each path vote for its best action (line 5).
  - Else the actions are sequentially played (line 6).
- The reward $y_k(t)$ is received (line 8).
- The decision stumps of each path c_θ are updated (lines 9-15).
- When the set of remaining variables of the decision stump corresponding to the path c_θ contains only one variable and the maximum depth D_θ is not reached (line 16), two new decision stumps corresponding to the values 0,1 of the selected variable are allocated (lines 18-20). The random set of remaining variables $\mathcal{V}_{c_\theta}$, the counts, and the estimated means are initialized in function NewPath.

To take into account the L decision trees of maximum depth D_θ, irrelevant variables are eliminated using a slight modification of inequality (1). A possible next variable x_i is eliminated when:

$$\hat{\mu}^{i'} | c_\theta - \hat{\mu}^i | c_\theta + \epsilon \geq 4 \sqrt{\frac{1}{2 t_{c_\theta,k}} \log \frac{4 \times 2^D K M D_\theta L t^2_{c_\theta,k}}{\delta}} \, , \tag{3}$$

where $i' = \arg\max_{i \in \mathcal{V}_{c_\theta}} \hat{\mu}^i | c_\theta$, and $t_{c_\theta,k}$ is the number of times the path c_θ and the action k have been observed. To take into account the L decision trees, irrelevant actions are eliminated using a slight modification of inequality (2):

$$\hat{\mu}_{k'|c_\theta} - \hat{\mu}_{k|c_\theta} + \epsilon \geq 2 \sqrt{\frac{1}{2 t_{c_\theta,i,v,k}} \log \frac{4 \times 2^D K L t^2_{c_\theta,i,v,k}}{\delta}} \, , \tag{4}$$

where $k' = \arg\max_k \hat{\mu}_{k|c_\theta}$, and $t_{c_\theta,i,v,k}$ is the number of times the action k has been played when the path c_θ, and the value v of the variable i have been observed.

Theorem 3: *when $K \geq 2$, $M \geq 2$, and $\epsilon = 0$, the sample complexity needed by* BANDIT FOREST *learning to obtain the optimal random forest of size L with a probability at least $1 - \delta$ is:*

$$t^* = 2^D \left(\frac{64K}{\Delta_1^2} \log \frac{4KMDL}{\delta \Delta_1} + \frac{64K}{\Delta_2^2} \log \frac{4LK}{\delta \Delta_2} \right) \, ,$$

where $\Delta_1 = \min_{\theta, c^*_\theta, i \neq i^*} P(c^*_\theta) \left(\mu^{i^*} | c^*_\theta - \mu^i | c^*_\theta \right)$, $\Delta_2 = \min_{\theta, c^*_\theta, k \neq k^*} P(c^*_\theta) \left(\mu_{k^*} | c^*_\theta - \mu_k | c^*_\theta \right)$, *and* $D = \max_\theta D_\theta$.

The dependence of the sample complexity on the depth D is exponential. This means that like all decision trees, BANDIT FOREST is well suited for cases, where there is a small subset of relevant variables belonging to a large set of variables ($D << M$). This usual restriction of local models is not a problem for a lot of applications, where one can build thousands of contextual variables, and where only a few of them are relevant.

Theorem 4: *There exists a distribution $D_{x,y}$ such that any algorithm finding the optimal random forest of size L has a sample complexity of at least:*

$$\Omega\left(2^D\left[\frac{1}{\Delta_1^2}+\frac{1}{\Delta_2^2}\right]K\log\frac{1}{\delta}\right)$$

Theorem 4 shows that BANDIT FOREST algorithm is near optimal. The result of this analysis is supported by empirical evidence in the next section.

Remark 4: We have chosen to analyze BANDIT FOREST algorithm in the case of $\epsilon = 0$ in order to simplify the concept of the optimal policy. Another way is to define the set of ϵ-optimal *random forests*, built with decision stumps which are optimal up to an ϵ approximation factor. In this case, the guarantees are given with respect to the worst policy of the set. When $\epsilon = 0$, this set contains only the optimal *random forest* of size L given the values of Θ.

5 Experimentation

In order to illustrate the theoretical analysis with reproducible results on large sets of contextual variables, we used three datasets from the *UCI Machine Learning Repository* (*Forest Cover Type*, *Adult*, and *Census1990*). We recoded each continuous variable using *equal frequencies* into 5 binary variables, and each categorical variable into disjunctive binary variables. We obtained 94, 82 and 255 binary variables, for *Forest Cover Type*, *Adult* and *Census1990* datasets respectively. For *Forest Cover Type*, we used the 7 target classes as the set of actions. For *Adult*, the categorical variable *occupation* is used as a set of 14 actions. For *Census1990*, the categorical variable *Yearsch* is used as a set of 18 actions. The gain of policies was evaluated using the class labels of the dataset with a reward of 1 when the chosen action corresponds to the class label and 0 otherwise. The datasets, respectively composed of 581000, 48840 and 2458285 instances, were shuffled and played in a loop to simulate streams. In order to introduce noise between loops, at each time step the value of each binary variable has a probability of 0.05 to be inverted. Hence, we can consider that each context-reward vector is generated by a stationary random process. We set the time horizon to 10 millions of iterations. The algorithms are assessed in terms of accumulated regret against the optimal *random forest* of size 100. The optimal *random forest* is obtained by training a *random forest* of size 100 not limited by depth on the whole dataset with full information feedback and without noise.

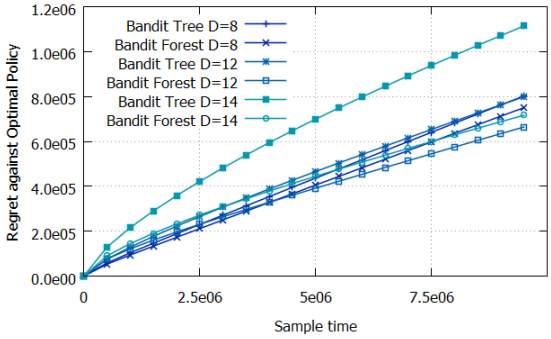


Figure 1: The accumulated regret of BANDIT FOREST and BANDIT TREE with different depths against the optimal policy averaged over ten trials on *Forest Cover Type* dataset played in a loop with noisy inputs.

To effectively implement BANDIT FOREST, we have done two modifications of the analyzed algorithms. Firstly, the Round-robin function is replaced by an uniform random draw from the union of the remaining actions of the paths. Secondly, the rewards are normalized using *Inverse Propensity Scoring* (see Horvitz and Thompson (1952)): the obtained reward is divided by the probability to draw the played action. First of all, notice that the regret curves of BANDIT FOREST algorithm are far from those of explore then exploit approaches: the regret is gradually reduced over time (see Figure 1). Indeed, BANDIT FOREST algorithm uses a *localized explore then exploit* approach: most of paths, which are unlikely, may remain in exploration state, while the most frequent ones already vote for their best actions. The behavior of the algorithm with regard to number of trees L is simple: the higher L, the greater the performances, and the higher the computational cost (see Figure 3). To analyze the sensitivity to depth of BANDIT FOREST algorithm, we set the maximum depth and we compared the performances of a single tree without randomization (BANDIT TREE) and of BANDIT FOREST with $L = 100$. The trees of the forest are randomized at each node with different values of ϵ (between 0.4 and 0.8), and with different sets of available splitting variables (random subset of 80% of remaining variables). For each tested depth, a significant improvement is observed thanks to the vote of randomized trees (see Figure 1). Moreover, BANDIT FOREST algorithm appears to be less sensible to depth than a single tree: the higher the difference in depth, the higher the difference in performance.

To compare with state-of-the-art, the trees in the forest are randomized with different values of the parameters D (between 10 and 18). On the three datasets, BANDITRON (Kakade et al (2008)) is clearly outperformed by the other

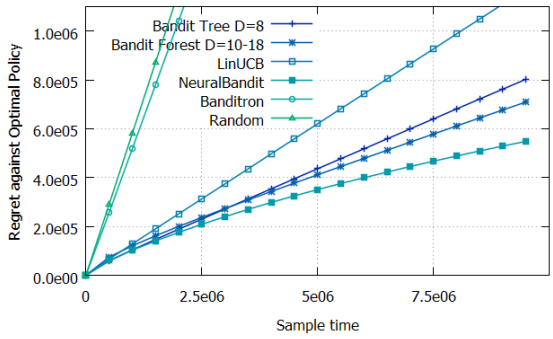


Figure 2: The accumulated regret against the optimal policy averaged over ten trials for the dataset *Forest Cover Type* played in a loop with noisy inputs.

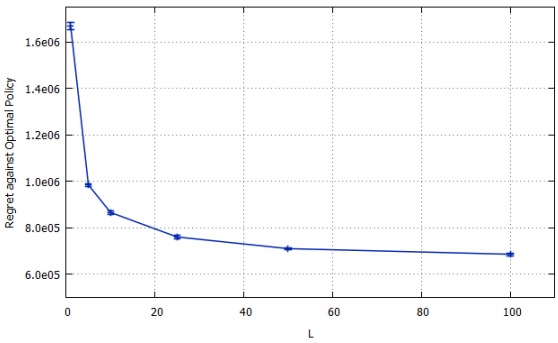


Figure 3: Sensitivity to the size of BANDIT FOREST on *Census 1990* dataset.

tested algorithms (see Figures 2, 4, 5). NEURAL BANDIT (Allesiardo et al (2014)) is a Committee of Multi-Layer Perceptrons (MLP). Due to the non convexity of the error function, MLP trained with the back-propagation algorithm (Rumelhart et al (1986)) does not find the optimal solution. That is why finding the regret bound of such model is still an open problem. However, since MLPs are universal approximators (Hornik et al (1989)), NEURAL BANDIT is a very strong baseline. It outperforms LINUCB (Li et al (2010)) on *Forest Cover Type* and on *Adult*. BANDIT FOREST clearly outperforms LINUCB and BANDITRON on the three datasets. In comparison to NEURAL BANDIT, BANDIT FOREST obtains better results on *Census 1990* and *Adult*, and it is outperformed on *Forest Cover Type*, where the number of strong features seems to be high. Finally as shown by the worst case analysis, the risk to use BANDIT FOREST on a lot of optimization problems is controlled, while NEURAL BANDIT, which has no theoretical guaranty, can obtain poor performances on some problems, such as *Census 1990* where it is outperformed by the linear solution LINUCB. This uncontrolled risk increases with the number of actions, since the probability to obtain a non robust MLP linearly increases with the number of actions.

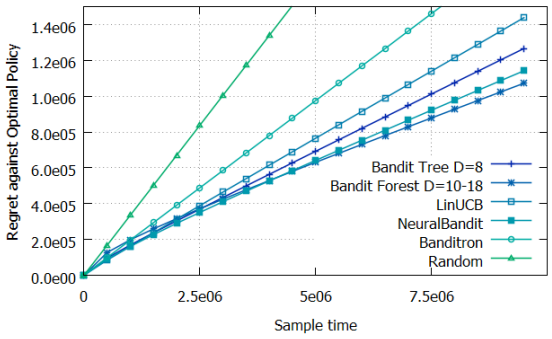


Figure 4: The accumulated regret against the optimal policy averaged over ten trials for the dataset *Adult* played in a loop with noisy inputs.

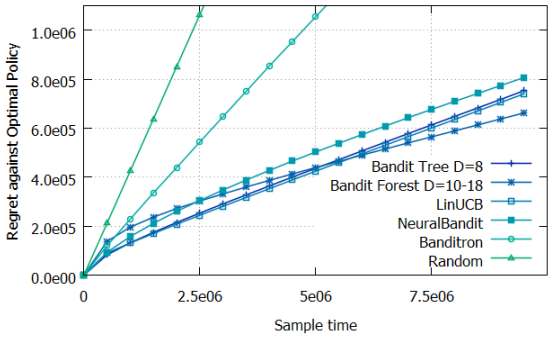


Figure 5: The accumulated regret against the optimal policy averaged over ten trials for the dataset *Census1990* played in a loop with noisy inputs.

6 Conclusion

We have shown that the proposed algorithm is optimal up to logarithmic factors with respect to a strong reference: a *random forest* built knowing the joint distribution of the contexts and rewards. In the experiments, BANDIT FOREST clearly outperforms LINUCB, which is a strong baseline with known regret bound, and performs as well as NEURAL BANDIT, for which we do not have theoretical guaranty. Finally, for applications where the number of strong features is low in comparison to the number of possible features, which is often the case, BANDIT FOREST shows valuable properties:

- its sample complexities have a logarithmic dependence on the number of contextual variables, which means that it can process a large amount of contextual variables with a low impact on regret,
- its low computational cost allows to process efficiently infinite data streams,
- like all decision tree algorithms, it is well suited to deal with non linear dependencies between contexts and rewards.

References

Abbasi-Yadkori Y., Pál D., and Szepesvári, C.: Improved Algorithms for Linear Stochastic Bandits, NIPS, 2011.

Agrawal, A., Hsu, D., Kale, S., Langford, J., Li, L., Schapire, R. E.: Taming the Monster: A Fast and Simple Algorithm for Contextual Bandits, ICML, 2014.

Allesiardo, E., Féraud, R., Bouneffouf, D.: A Neural Networks Committee for the Contextual Bandit Problem, ICONIP, 2014.

Auer, P., Cesa Bianchi, N., Fischer, P.: Finite-time Analysis of the Multiarmed Bandit Problem, Machine Learning,47, 235-256, 2002.

Auer, P., Cesa-Bianchi, N., Freund, Y., Schapire, R. E.: The nonstochastic multiarmed bandit problem, SIAM J. COMPUT., 32 48-77, 2002.

Baransi, Akram and Maillard, Odalric-Ambrym and Mannor, Shie: Sub-sampling for Multi-armed Bandits, Machine Learning and Knowledge Discovery in Databases, 8724, 115-131, 2014.

Biau, G.: Analysis of Random Forest Model, JMLR, 13:1063-1095, 2012.

Breiman, L., Friedman, J. H., Olshen, R. A., Stone, C. J.: Classification and regression trees, Monterey, CA: Wadsworth & Brooks Cole Advanced Books & Software, 1984.

Breiman, L.: Random Forest, Machine Learning, 45:5-32, 2001.

Bubeck, S., Wang, T., Stoltz: Pure exploration in multi-armed bandits problems, COLT, 2009.

Chakrabarti, D., Kumar, R., Radlinski, F., Upfal, E.: Mortal multi-armed bandits, NIPS, pp. 273-280, 2008.

Dudík, M., Hsu, D., Kale, S., Karampatziakis, N., Langford, J., Reyzin, L., Zhang, T.:Efficient Optimal Learning for Contextual Bandits, UAI, 169-178, 2011.

Domingos, P., Hulten, G.: Mining high-speed data streams, In: KDD, 71-80, 2000.

Even-Dar, E., Mannor, S., Mansour Y.: PAC Bounds for Multi-armed Bandit and Markov Decision Processes, COLT, 255-270, 2002.

Even-Dar, E., Mannor, Mansour, Y.: Action Elimination Stopping Conditions for the Multi-Armed Bandit and Reinforcement Learning Problems, JMLR 7:1079-1105, 2006.

Féraud, R., Urvoy, T.: A stochastic bandit algorithm for scratch games, ACML, 25:129-145, 2012.

Féraud, R., Urvoy, T.: Exploration and Exploitation of Scratch Games, Machine Learning, 92:377-401, 2013.

Fernàndez-Delgado, M., Cernadas, E., Barro, S.: Do we Need Hundreds of Classifiers to Solve Real World Classification Problems ?, JMLR 15:3133-3181, 2014.

Hoeffding, W.: Probability inequalities for sums of bounded random variables. J. Amer. Statist. Assoc. 58:13-30, 1963.

Hornik, K., Stinchcombe, M., White, H.: Multilayer feedforward networks are universal approximators, Neural Networks. 2:359-366, 1989.

Horvitz, D. G., Thompson, D. J.: A generalization of sampling without replacement from a finite universe. Journal of the American Statistical Association, 47(260): 663-685, 1952.

Kakade, S. M., Shalev-Shwartz, S., Tewari, A.: Efficient Bandit Algorithms for Online Multiclass Prediction, ICML, 2008.

Kaufman, E., Korda, N., Munos, R.: Thomson sampling: An asymptotically optimal finite time analysis, COLT, 2012.

Kleinberg, R. D., Niculescu-Mizil, Sharma, T.: Regrets bounds for sleeping experts and bandits, COLT, 2008.

Kocsis, L., Szépesvari, C.: Bandit based monte-carlo planning, ECML, 2006.

Langford, J., Zhang, T.: The epoch-greedy algorithm for contextual multi-armed bandits, NIPS, 2007.

Li, L., Chu, W., Langford, J., Schapire, R. E.: A Contextual-Bandit Approach to Personalized News Article Recommendation, Nineteenth International Conference on World Wide Web, 2010.

Mannor, S., and Tsitsiklis, J. N.: The Sample Complexity of Exploration in the Multi-Armed Bandit Problem, JMLR, 2004

Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Parallel distributed processing: Explorations in the microstructure of cognition, vol. 1, pp. 318-362. MIT Press, Cambridge, 1986.

Soare, M., Lazaric, L., Munos, R.: Best-Arm Identification in Linear Bandits, NIPS, 2014.

Thompson, W.R.: On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. Biometrika 25:285-294, 1933.

Urvoy, T., Clérot, F., Féraud, R., Naamane, S.: Generic Exploration and K-armed Voting Bandits, ICML, 2013.

Valiant, L.: A theory of the learnable, Communications of the ACM, 27, 1984.

7 Appendix

7.1 Notations

In order to facilitate the reading of the paper, we provide below (see Table 2) the list of notations.

Table 2: Notations

notation	description
K	number of actions
M	number of contextual variables
D_θ	maximum depth of the tree θ
L	number of trees
T	time horizon
$\mathcal{A}$	set of actions
$\mathcal{V}$	set of variables
$\mathcal{S}$	set of remaining variables
$\mathbf{x}$	context vector $\mathbf{x} = (x_1, \ldots, x_M)$
$\mathbf{y}$	reward vector $\mathbf{y} = (y_1, \ldots, y_K)$
k_t	action chosen at time t
c_θ	context path the tree θ, $c_\theta = (x_{i_1}, v_{i_1}), ..., (x_{i_{d_\theta}}, v_{i_{d_\theta}})$
d_θ	current depth of the context path c_θ
μ_k	expected reward of action k, $\mu_k = \mathbb{E}_{D_y}[y_k]$
$\mu_k^i \vert v$	expected reward of action k conditioned to $x_i = v$, $\mu_k^i \vert v = \mathbb{E}_{D_y}[y_k \cdot \mathbb{1}_{x_i = v}]$
$\mu_{k,v}^i$	expected reward of action k and $x_i = v$, $\mu_{k,v}^i = \mathbb{E}_{D_{x,y}}[y_k \cdot \mathbb{1}_{x_i = v}]$
μ^i	expected reward for the use of the variable x_i to select the best actions
δ	probability of error
ϵ	approximation error
Δ_1	minimum of difference with the expected reward of the best action μ_{k^*} and the expected reward of a given action k: $\Delta_1 = \min_{k \neq k^*}(\mu_{k^*} - \mu_k)$
Δ_2	minimum of difference with the best variable expected reward μ^{i^*} and the expected reward for other variables: $\Delta_2 = \min_{i \neq i^*}(\mu^{i^*} - \mu^i)$
t^*	sample complexity of the decision stump

7.2 Lemma 1

Proof. We cannot use directly Hoeffding inequality (see Hoeffding (1963)) to bound the estimated gains of the use of variables. The proof of Lemma 1 overcomes this difficulty by bounding each estimated gain μ^i by the sum of the bounds over the values of the expected reward of the best action $\mu_{k^*,v}^i$ (inequality 7). From Hoeffding's inequality, at time t we have:

$$\mathbb{P}\left(\left|\hat{\mu}_{k,v}^i - \mu_{k,v}^i\right| \geq \alpha_{t_k}\right) \leq 2\exp(-2\alpha_{t_k}^2 t_k) = \frac{\delta}{4KMt_k^2},$$

where $\alpha_{t_k} = \sqrt{\frac{1}{2t_k}\log\frac{4KMt_k^2}{\delta}}$.

Using Hoeffding's inequality on each time t_k, applying the union bound and then $\sum 1/t_k^2 = \pi^2/6$, the following inequality holds for any time t with a probability $1 - \frac{\delta\pi^2}{12KM}$:

$$\hat{\mu}_{k,v}^i - \alpha_{t_k} \leq \mu_{k,v}^i \leq \hat{\mu}_{k,v}^i + \alpha_{t_k} \tag{5}$$

If the inequality (5) holds for the actions $k' = \arg\max_k \hat{\mu}^i_{k,v}$, and $k^* = \arg\max_k \mu^i_{k,v}$, we have:

$$\begin{aligned} &\hat{\mu}^i_{k',v} - \alpha_{t_k} \leq \mu^i_{k',v} \leq \mu^i_{k^*,v} \leq \hat{\mu}^i_{k^*,v} + \alpha_{t_k} \leq \hat{\mu}^i_{k',v} + \alpha_{t_k} \\ &\Rightarrow \hat{\mu}^i_{k',v} - \alpha_{t_k} \leq \mu^i_{k^*,v} \leq \hat{\mu}^i_{k',v} + \alpha_{t_k} \end{aligned} \tag{6}$$

If the previous inequality (6) holds for all values v of the variable x_i, we have:

$$\begin{aligned} &\sum_{v\in\{0,1\}} \left(\hat{\mu}^i_{k',v} - \alpha_{t_k}\right) \leq \sum_{v\in\{0,1\}} \mu^i_{k^*,v} \leq \sum_{v\in\{0,1\}} \left(\hat{\mu}^i_{k',v} + \alpha_{t_k}\right) \\ &\Leftrightarrow \hat{\mu}^i - 2\alpha_{t_k} \leq \mu^i \leq \hat{\mu}^i + 2\alpha_{t_k} \end{aligned} \tag{7}$$

If the previous inequality holds for $i' = \arg\max_i \hat{\mu}^i$, we have:

$$\hat{\mu}^{i'} - 2\alpha_{t_k} \leq \mu^{i'} \leq \mu^{i^*} \tag{8}$$

As a consequence, the variable x_i cannot be the best one when:

$$\hat{\mu}^i + 2\alpha_{t_k} \leq \hat{\mu}^{i'} - 2\alpha_{t_k}\ , \tag{9}$$

Using the union bound, the probability of making an error about the selection on the next variable by eliminating each variable x_i when the inequality (9) holds is bounded by the sum for each variable x_i and each value v that the inequality 6 does not hold for k' and k^*:

$$\mathbb{P}\left(i^* \neq i'\right) \leq \sum_{i\in\mathcal{V}} \frac{K\delta\pi^2}{24KM} \leq \sum_{i\in\mathcal{V}} \frac{\delta}{M} \leq \delta \tag{10}$$

Now, we have to consider t^*_k, the number of steps needed to select the optimal variable. If the best variable has not been eliminated (probability $1-\delta$), the last variable x_i is eliminated when:

$$\hat{\mu}^{i^*} - \hat{\mu}^i \geq 4\alpha_{t^*_k}$$

The difference between the expected reward of a variable x_i and the best next variable is defined by:

$$\Delta_i = \mu^{i^*} - \mu^i$$

Assume that:

$$\Delta_i \geq 4\alpha_{t_k} \tag{11}$$

The following inequality holds for the variable x_i with a probability $1 - \frac{\delta}{KM}$:

$$\hat{\mu}^i - 2\alpha_{t_k} \leq \mu^i \leq \hat{\mu}^i + 2\alpha_{t_k}$$

Then, using the previous inequality in the inequality (11), we obtain:

$$(\hat{\mu}^{i^*} + 2\alpha_{t_k}) - (\hat{\mu}^i + 2\alpha_{t_k}) \geq \mu^{i^*} - \mu^i \geq 4\alpha_{t_k}$$

Hence, we have:

$$\hat{\mu}^{i^*} - \hat{\mu}^i \geq 4\alpha_{t_k}$$

The condition $\Delta_i \geq 4\alpha_{t_k}$ implies the elimination of the variable x_i. Then, we have:

$$\Delta_i \geq 4\alpha_{t_k}$$

$$\Rightarrow \Delta_i^2 \geq \frac{8}{t_k} \log \frac{4KMt_k^2}{\delta} \tag{12}$$

The time t_k^*, where all non optimal variables have been eliminated, is reached when the variable corresponding to the minimum of Δ_i^2 is eliminated.

$$\Rightarrow \Delta^2 \geq \frac{8}{t_k^*} \log \frac{4KMt_k^{*2}}{\delta} \,, \tag{13}$$

where $\Delta = \min_{i \neq i^*} \Delta_i$.

The inequality (13) holds for all variables i with a probability $1 - \delta$ for:

$$t_k^* = \frac{64}{\Delta^2} \log \frac{4KM}{\delta.\Delta} \tag{14}$$

Indeed, if we replace the value of t_k^* in the right term of the inequality (12), we obtain:

$$\begin{aligned}
&\frac{\Delta^2}{8 \log \frac{4KM}{\delta.\Delta}} \left(\log \frac{4KM}{\delta} + 2 \log \frac{64}{\Delta^2} + 2 \log \log \frac{4KM}{\delta.\Delta} \right) = \\
&\frac{\Delta^2}{8 \log \frac{4KM}{\delta.\Delta}} \left(\log \frac{4KM}{\delta} - 4 \log \Delta + 12 \log 2 + 2 \log \log \frac{4KM}{\delta.\Delta} \right) \leq \\
&\frac{\Delta^2}{8 \log \frac{4KM}{\delta.\Delta}} \left(4 \log \frac{4KM}{\delta.\Delta} + 12 \log 2 + 2 \log \log \frac{4KM}{\delta.\Delta} \right)
\end{aligned}$$

For $x \geq 13$, we have:

$$12 \log 2 + 2 \log \log x < 4 \log x$$

Hence, for $4KM \geq 13$, we have:

$$\begin{aligned}
&\frac{\Delta^2}{8 \log \frac{4KM}{\delta.\Delta}} \left(4 \log \frac{4KM}{\delta.\Delta} + 12 \log 2 + 2 \log \log \frac{4KM}{\delta.\Delta} \right) \leq \\
&\frac{\Delta^2}{8 \log \frac{4KM}{\delta.\Delta}} 8 \log \frac{4KM}{\delta.\Delta} = \Delta^2
\end{aligned}$$

Hence, we obtain:

$$t_k^* = \frac{64}{\Delta^2} \log \frac{4KM}{\delta} \text{ , with a probability } 1 - \delta$$

As the actions are chosen the same number of times (Round-robin function), we have $t = Kt_k$, and thus:

$$t^* = \frac{64K}{\Delta^2} \log \frac{4KM}{\delta} \text{ , with a probability } 1 - \delta$$

□

7.3 Lemma 2

Proof. Let $y_{k,v}^i$ be a bounded random variable corresponding to the reward of the action k when the value v of the variable i is observed. Let y^i be a random variable such that:

$$y^i = \max_k y_{k,v}^i$$

We have:

$$\mathbb{E}_{D_{x,y}}[y^i] = \mu^i$$

Each y^i is updated each step t_k when each action has been played once. Let Θ be the sum of the binary random variables $\theta_1, ..., \theta_{t_k}, ..., \theta_{t_k^*}$ such that $\theta_{t_k} = \mathbb{1}_{y^i(t_k) \geq y^j(t_k)}$. Let p_{ij} be the probability that the use of variable i leads to more rewards than the use of variable j. We have:

$$p_{ij} = \frac{1}{2} - \Delta_{ij} \text{ , where } \Delta_{ij} = \mu^i - \mu^j.$$

Slud's inequality (see Slud (1977)) states that when $p \leq 1/2$ and $t_k^* \leq x \leq t_k^*.(1-p)$, we have:

$$P(\Theta \geq x) \geq P\left(Z \geq \frac{x - t_k^*.p}{\sqrt{t_k^*.p(1-p)}}\right) , \tag{15}$$

where Z is a normal $\mathcal{N}(0,1)$ random variable.

To choose the best variable between i and j, one needs to find the time t_k^* where $P(\Theta \geq t_k^*/2) \geq \delta$. To state the number of trials t_k^* needed to estimate Δ_{ij}, we recall and adapt the arguments developed in Mousavi (2010). Using Slud's inequality (see equation 15), we have:

$$P(\Theta \geq t_k^*/2) \geq P\left(Z \geq \frac{t_k^*.\Delta_{ij}}{\sqrt{t_k^*.p_{ij}(1-p_{ij})}}\right) , \tag{16}$$

Then, we use the lower bound of the error function (see Chu (1955)):

$$P(Z \geq z) \geq 1 - \sqrt{1 - \exp\left(-\frac{z^2}{2}\right)}$$

Therefore, we have:

$$\begin{aligned} P(\Theta \geq t_k^*/2) &\geq 1 - \sqrt{1 - \exp\left(-\frac{t_k^*.\Delta_{ij}^2}{2p_{ij}(1-p_{ij})}\right)} \\ &\geq 1 - \sqrt{1 - \exp\left(-\frac{t_k^*.\Delta_{ij}^2}{p_{ij}}\right)} \\ &\geq \frac{1}{2}\exp\left(-\frac{t_k^*.\Delta_{ij}^2}{p_{ij}}\right) \end{aligned}$$

As $p_{ij} = 1/2 - \Delta_{ij}$, we have:

$$\log\delta = \log\frac{1}{2} - \frac{t_k^*.\Delta_{ij}^2}{1/2 - \Delta_{ij}} \geq \log\frac{1}{2} - 2t_k^*.\Delta_{ij}^2$$

Hence, we have:

$$t_k^* = \Omega\left(\frac{1}{\Delta_{ij}^2}\log\frac{1}{\delta}\right)$$

Then, we need to use the fact that as all the values of all the variables are observed when one action is played: the $M(M-1)/2$ estimations of bias are solved in parallel. In worst case, $\min_{ij} \Delta_{ij} = \min_j \Delta_{i^*j} = \Delta$. Thus any algorithm needs at least a sample complexity t^*, where:

$$t^* = K.t_k^* = \Omega\left(\frac{K}{\Delta^2}\log\frac{1}{\delta}\right)$$

□

7.4 Theorem 1

Proof. Lemma 1 states that the sample complexity needed to find the best variable is:

$$t_1^* = \frac{64K}{\Delta_1^2}\log\frac{4KM}{\delta\Delta_1}, \text{where} \quad \Delta_1 = \min_{i\neq i^*}(\mu^{i^*} - \mu^i)$$

Lemma 3 states that the sample complexity needed to find the optimal action for a value v of the best variable is:

$$t_{2,v}^* = \frac{64K}{\Delta_{2,v}^2}\log\frac{4K}{\delta\Delta_{2,v}}, \text{where} \quad \Delta_{2,v} = \min_{k\neq k^*}(\mu_{k^*,v}^{i^*} - \mu_{k,v}^{i^*}).$$

The sample complexity of decision stump algorithm is bounded by the sum of the sample complexities of variable selection and action elimination algorithms:

$$t^* = t_1^* + t_2^* \text{ , where } t_2^* = \max_v t_{2,v}^*.$$

□

7.5 Theorem 2

Proof. In worst case, all the values of variables have different best actions, and $K = 2M$. If an action is suppressed before the best variable is selected, the estimation of the mean reward of one variable is underestimated. In worst case this variable is the best one, and a sub-optimal variable is selected. Thus, the best variable has to be selected before an action be eliminated. The lower bound of the decision stump problem is the sum of variable selection and best arm identification lower bounds, stated respectively in Lemma 2 and Lemma 4.

□

7.6 Theorem 3

Proof. The proof of Theorem 3 uses Lemma 1 and Lemma 3. Using the slight modification of the variable elimination inequality proposed in section 3, Lemma 1 states that for each decision stump, we have:

$$\mathbb{P}\left(i_{d_\theta}^*|c_\theta \neq i'_{d_\theta}|c_\theta\right) \leq \frac{\delta}{2^{D_\theta}L}$$

For the action corresponding to the path c_θ, Lemma 3 states that:

$$\mathbb{P}(k \neq k^*) \leq \frac{\delta}{2^{D_\theta}L}$$

From the union bound, we have:

$$\mathbb{P}(\exists c_\theta \text{such that} c_\theta \neq c_\theta^*) \leq \delta \text{ and } \mathbb{P}(\exists k \text{such that} k \neq k^*) \leq \delta$$

Using Lemma 1 and Lemma 3, and summing the sample complexity of each 2^{D_θ} variable selection tasks and the sample complexity of each 2^{D_θ} action selection tasks, we bound the sample complexity of any tree θ by:

$$t^* \leq 2^D \frac{64K}{\Delta_1^2} \log \frac{4KMDL}{\delta\Delta_1} + 2^D \frac{64K}{\Delta_2^2} \log \frac{4KL}{\delta\Delta_2} ,$$

where $D = \max D_\theta$.

□

7.7 Theorem 4

Proof. To build a decision tree of depth D_θ, any greedy algorithm needs to solve $\sum_{d<D_\theta} 2^d = 2^{D_\theta}$ variable selection problems (one per node), and 2^{D_θ} action selection problems (one per leaf). Then, using Lemma 2 and Lemma 4, any greedy algorithm needs a sample complexity of at least:

$$t^* \geq \Omega\left(2^D \left[\frac{1}{\Delta_1^2} + \frac{1}{\Delta_2^2}\right] K \log \frac{1}{\delta}\right)$$

□

7.8 Additional experimental results

We provide below (see Table 3) the classification rates to compare the asymptotical performances of each algorithm, and the processing times.

Table 3: Summary of results on the datasets played in a loop. The regret against the optimal *random forest* is evaluated on ten trials. Each trial corresponds to a random starting point in the dataset. The confidence interval is given with a probability 95%. The classification rate is evaluated on the last 100000 contexts. The mean running time was evaluated on a simple computer with a quad core processor and 6 GB of RAM.

Algorithm	Regret	Classification rate	Running time
Forest Cover Type, action: Cover Type (7 types)			
BANDITRON	$1.99\ 10^6 \pm 10^5$	49.1%	10 min
LINUCB	$1.23\ 10^6 \pm 10^3$	60%	360 min
NEURALBANDIT	$0.567\ 10^6 \pm 2.10^4$	68%	150 min
BANDIT TREE $D = 8$	$0.843\ 10^6 \pm 2.10^5$	64.2%	5 min
BANDIT FOREST $D10 - 18$	$0.742\ 10^6 \pm 5.10^4$	65.8%	500 min
Adult, action: occupation (14 types)			
BANDITRON	$1.94\ 10^6 \pm 3.10^4$	21.1%	20 min
LINUCB	$1.51\ 10^6 \pm 4.10^4$	25.7%	400 min
NEURALBANDIT	$1.2\ 10^6 \pm 10^5$	29.6%	140 min
BANDIT TREE $D = 8$	$1.33\ 10^6 \pm 10^5$	27.9%	4 min
BANDIT FOREST $D10 - 18$	$1.12\ 10^6 \pm 7.10^4$	31%	400 min
Census1990, action: Yearsch (18 types)			
BANDITRON	$2.07\ 10^6 \pm 2.10^5$	27%	26 min
LINUCB	$0.77\ 10^6 \pm 5.10^4$	40.3%	1080 min
NEURALBANDIT	$0.838\ 10^6 \pm 10^5$	41.7%	300 min
BANDIT TREE $D = 8$	$0.78\ 10^6 \pm 2.10^5$	41%	10 min
BANDIT FOREST $D10 - 18$	$0.686\ 10^6 \pm 5.10^4$	43.2%	1000 min

References

Chu, J. T.: On bounds for the normal integral. Biometrika 42:263-265, 1955.

Mousavi, N.: How tight is Chernoff bound ?
https://ece.uwaterloo.ca/ nmousavi/Papers/Chernoff-Tightness.pdf

Slud, E. V.: Distribution Inequalities for the Binomial Law. Ann. Probab.,5(3):404-412, 1977.